

Wick & Wait – Strategie in der Vault anlegen & manuell testen

Pecunity Strategy Builder · BSC · PancakeSwap V3 · Stand: 14.06.2026

Diese Anleitung führt dich als **User** Schritt für Schritt durch den kompletten Aufbau der Wick-&-Wait-Concentrated-Liquidity-Strategie auf einer Vault – aus drei Automations – und durch einen manuellen End-to-End-Test auf dem Fork.

Inhalt

1. Was die Strategie macht
2. Voraussetzungen
3. Grundkonzepte (Context-Slots, Trigger vs. Aktion, Public vs. Owner-only)
4. Beispiel-Parameter (BTCB/USDT)
5. Schritt 0 – Vault vorbereiten (Deposit + Gas-Deposit + Pool-Oracle)
6. Schritt 1 – Context-Slots anlegen
7. Schritt 2 – Automation „Entry“
8. Schritt 3 – Automation „Rebalance“
9. Schritt 4 – Automation „Auto-Compound“
10. Schritt 5 – Manueller End-to-End-Test
11. Troubleshooting
12. Anhang – Feld-Referenz pro Step

1 · Was die Strategie macht

Die Strategie verdient Trading-Gebühren aus einer *konzentrierten* Liquiditätsposition und vermeidet unnötiges, teures Rebalancing. Verlässt der Preis die Range, wird **nicht sofort** reagiert – erst wenn der Preis lange genug draußen bleibt (TWAP-bestätigt). Kurze „Wicks“ werden ignoriert.

Sie besteht aus **drei Automations** auf derselben Vault, die sich eine Position teilen:

Automation	Trigger	Tut	Typ
Entry	– (manuell)	Eröffnet die Position aus dem Deposit-Token: sized on-chain auf das Zielverhältnis und mintet.	Owner-only
Rebalance	Wick & Wait (TWAP + Cooldown)	Erntet die Fees, schließt die Position bei anhaltendem Range-Bruch, sized die freien Token neu, öffnet um den neuen Preis.	Public
Auto-Compound	Interval	Erntet regelmäßig die Fees, füllt die Gas-Reserve auf und legt den Rest in dieselbe Position nach.	Public

2 · Voraussetzungen

- **Dienste laufen:** Backend (`:3001`), Frontend (`:5173`), Fork-Node (`:8545`).
- **Contracts deployed & geseedet** auf dem Fork – inkl. der neuen Steps `WickWaitRebalanceCondition` und `PancakeSwapV3SwapToRangeRatioAction` :

```
cd packages/contracts && npx hardhat run scripts/deploy-defi-actions.ts --network localhost
cd ../backend && pnpm db:seed
```

Prüfen: im Editor erscheinen unter `+ Add Step` die Steps „Wick & Wait Rebalance“ und „PancakeSwap V3 Swap to Range Ratio“.

- **Wallet** verbunden, Owner der Vault.
- **Kapital** im Deposit-Token (z. B. USDT) zum Einzahlen.

3 · Grundkonzepte

Context-Slots

Geteilter Speicher der Vault, über den Steps Werte austauschen. Die Position-NFT-ID wird von *Mint* in einen Slot geschrieben und von *Rebalance*, *Collect*, *Increase* und der *Wick-&-Wait-Condition* wieder gelesen. Deshalb teilen sich alle drei Automations denselben Slot.

Trigger (Condition) vs. Aktion (Action)

Eine Automation startet optional mit **einer Condition** (Step 0) als Auslöser, danach folgen Aktionen. Die Steps werden per Kante (Edge) **out → in** verkettet.

Public vs. Owner-only

Hat eine Automation einen öffentlichen Trigger, ist sie **Public** – ein *Keeper* führt sie aus, sobald der Trigger erfüllt ist (dafür braucht die Vault einen **Gas-Deposit**). Eine reine Aktions-Automation ohne Trigger (wie *Entry*) ist **Owner-only** und wird vom Owner über den **Execute**-Button manuell ausgeführt.

4 · Beispiel-Parameter (BTCB/USDT)

Parameter	Wert im Beispiel	Hinweis
Pool / Token A / Token B	BTCB & USDT	Eine Seite = Deposit-Token
Fee Tier	0,05% (500)	der validierte „tiefe“ Pool
Range Width (RANGE)	±10%	In Entry & Rebalance identisch!
TWAP Window (W)	30 min (Balanced)	Konservativ 1h / Balanced 30m / Aggressiv 10m
Cooldown	3 Tage	7d / 3d / 1d
Compound-Interval	1 Woche	frei wählbar

Wichtig: Die *Range Width* (±%) muss in der *Swap-to-Range-Ratio*-Aktion und der direkt folgenden *Mint*-Aktion **exakt gleich** sein – sonst sized der Swap auf ein anderes Verhältnis als die Position. Das gilt für Entry und Rebalance.

5 · Schritt 0 – Vault vorbereiten

1. **Vault öffnen:** Dashboard → deine Vault (Route `/vault/<adresse>`).
2. **Kapital einzahlen:** Karte *Deposit* → Betrag im Deposit-Token (z. B. USDT) → bestätigen (Approve + Deposit). Die Position entsteht später daraus.
3. **Gas-Deposit einrichten:** Karte *Gas Deposit* → eine positive *Zielreserve* (`minFeeDeposit`) setzen und initial einzahlen. Nötig, damit der Keeper die `Public`-Automations (Rebalance, Auto-Compound) ausführt — sonst revert `InsufficientFeeDeposit`. Die Reserve sinkt mit jedem Keeper-Lauf; der *Fee-Deposit*-Schritt im Auto-Compound (Schritt 4) füllt sie automatisch wieder auf.
4. **Pool-Oracle (TWAP) vorbereiten:** Der Pool muss genug Preis-Beobachtungen für das gewählte Fenster `W` haben, sonst *revertet* die Wick-&-Wait-Condition (sichtbar als nicht feuender Trigger). Einmalig die Observation-Cardinality erhöhen (`increaseObservationCardinalityNext`). Auf dem Fork erledigt das die Aufwärm-Phase des Preis-Treiber-Skripts (siehe Schritt 5).

6 · Schritt 1 – Context-Slots anlegen

Lege die zwei geteilten Slots **vor** dem Bauen der Automations an, damit du sie in den Step-Feldern auswählen kannst. Im Editor → rechte *Side-Panel* → Bereich *Context* → `+ Neue Context-Variable`.

Name	Typ	Zweck
<code>LP_POSITION_SLOT</code>	<code>uint256</code>	Hält die Position-NFT-ID. Mint schreibt sie; Rebalance/Collect/Increase/Condition lesen sie.
<code>LAST_REBALANCE_SLOT</code>	<code>uint256</code>	Zeitstempel des letzten Rebalance (Cooldown). Nur die Wick-&-Wait-Condition liest/schreibt ihn.

Slots sind vault-weit – einmal angelegt, in allen drei Automations auswählbar. Den Editor erreichst du über *Vault* → *Automations* → `Create Automation`.

7 · Schritt 2 – Automation „Entry“ Owner-only

Vault → Automations → `Create Automation`. Name oben eintragen: „*Wick&Wait – Entry*“. Steps über `+ Add Step` hinzufügen, dann verbinden.

Swap to Range Ratio → LP Mint

Step 1 · PancakeSwap V3 Swap to Range Ratio

Feld	Wert
Token A	USDT (Deposit-Token)
Token B	BTCB
Fee Tier	0,05% (500)
Range Width	Button <code>±10%</code> (oder eigenen % tippen)

Liest den Live-Preis, rechnet das Ziel-Verhältnis für die Range aus und tauscht den überrepräsentierten Token. Sizing passiert **on-chain zur Ausführungszeit**.

Step 2 · PancakeSwap V3 LP Mint

Feld	Wert
Token A / Token B	USDT / BTCB (gleich wie oben)
Fee Tier	0,05% (500)
Price Range	<i>Preset width</i> → <code>±10%</code> (identisch zu Step 1)
Amount A	Toggle „ <i>Volles Guthaben</i> “ aktivieren
Amount B	Toggle „ <i>Volles Guthaben</i> “ aktivieren

Position Token-ID to Context Slot

LP_POSITION_SLOT

Verbinden: Kante vom **out** -Handle von *Swap to Range Ratio* zum **in** -Handle von *LP Mint* ziehen. Dann **Deploy** → **Confirm & Deploy** (1 Transaktion; ggf. eine zweite „Set Context“-Tx, wenn der Slot neu initialisiert wird).

Da Entry keinen Trigger hat, ist sie **Owner-only**: in der Automations-Liste erscheint Status *Manual* und ein **Execute** -Button.

8 · Schritt 3 – Automation „Rebalance“ Public

Neue Automation, Name „Wick&Wait – Rebalance“.

Wick & Wait Rebalance → Collect → Decrease Liquidity → Swap to Range Ratio → LP Mint

Step 1 · Wick & Wait Rebalance (Trigger / Condition)

Feld	Wert
Position Token-Id Slot	LP_POSITION_SLOT
TWAP Window	30 minutes (Balanced)
Rebalance Cooldown	3 days
Last-Rebalance Slot	LAST_REBALANCE_SLOT

Feuert nur, wenn der TWAP-Tick das Range-Fenster für **W** verlassen hat *und* der Cooldown abgelaufen ist. Kurze Wicks bewegen den TWAP nicht heraus.

Step 2 · PancakeSwap V3 Collect

Feld	Wert
Position Token-ID from Context Slot	LP_POSITION_SLOT

Erntet zuerst die aufgelaufenen Fees in die Vault — **vor** dem Entnehmen des Prinzips.

Step 3 · PancakeSwap V3 Decrease Liquidity

Feld	Wert
Position Token-ID from Context Slot	LP_POSITION_SLOT
Percentage to Remove	100

Entfernt 100% und liefert das freie Prinzipal in die Vault.

Warum erst Collect, dann Decrease? Die Decrease-Aktion bündelt intern bereits ein `collect(max,max)`. Der separate Collect davor erntet die Fees, bevor das Prinzipal entnommen wird — Decrease danach holt den Rest. (Decrease → Collect hätte den zweiten Schritt zum No-op gemacht.)

Step 4 · PancakeSwap V3 Swap to Range Ratio

Feld	Wert
Token A / Token B	USDT / BTCB
Fee Tier	0,05% (500)
Range Width	±10% — gleich wie Entry

Step 5 · PancakeSwap V3 LP Mint

Feld	Wert
------	------

Token A / Token B / Fee	USDT / BTCTB / 500
Price Range	Preset width ±10%
Amount A / Amount B	je „Volles Guthaben“
Position Token-ID to Context Slot	LP_POSITION_SLOT (überschreibt die ID)

Verbinden: **Trigger** → **Collect** → **Decrease** → **Swap to Range Ratio** → **Mint** . Dann **Deploy** . Nach dem Deploy in der Liste **Activate** klicken.

Beide Token & Dust: Nach dem Schließen hält die Vault *beide* Token (Fees + Prinzipal). *Swap to Range Ratio* bringt sie aufs Ziel-Verhältnis der neuen Range, *Mint (Volles Guthaben)* providet sie — durch Single-Pass-Sizing bleibt aber ein kleiner **Dust**-Rest eines Tokens in der Vault (es wird also nicht zwingend alles bereitgestellt). Der Rest bleibt liegen und wird beim nächsten Compound/Rebalance mitgenutzt. Gilt analog für den Entry.

9 · Schritt 4 – Automation „Auto-Compound“ Public

Neue Automation, Name „Wick&Wait – Auto-Compound“.

Interval Condition → Collect → Fee Deposit → Increase Liquidity

Step 1 · Interval Condition (Trigger)

Feld	Wert
Interval	1 weeks
Start Time	jetzt (Default)

Der Zeit-Slot wird automatisch angelegt und nach jeder Ausführung drifftfrei weitergesetzt.

Step 2 · PancakeSwap V3 Collect

Feld	Wert
Position Token-ID from Context Slot	LP_POSITION_SLOT

Step 3 · Fee Deposit (Gas-Reserve auffüllen)

Feld	Wert
Fee Registry	FeeRegistry-Adresse (aus dem Deploy)
Token	Deposit-Token (z. B. USDT)
Top-up Amount	Toggle „Bis Zielreserve auffüllen“ aktivieren

Füllt die Gas-Kompensations-Reserve aus dem Vault-Guthaben auf `minFeeDeposit` auf (No-op, wenn schon $\geq$ Ziel). So bleibt die Vault zahlungsfähig für den Keeper — ohne separate Wartungs-Automation. Läuft **vor** Increase, damit zuerst Gas gesichert ist.

Step 4 · PancakeSwap V3 Increase Liquidity

Feld	Wert
Token A / Token B	USDT / BTCB
Position Token-ID from Context Slot	LP_POSITION_SLOT
Amount A / Amount B	je „Volles Guthaben“ (legt die restlichen Fees nach)

Verbinden: Trigger → Collect → Fee Deposit → Increase . Deploy → Activate .

10 · Schritt 5 – Manueller End-to-End-Test

Reihenfolge zwingend: **Entry** **zuerst** – er füllt `LP_POSITION_SLOT` . Ohne gültige Position-ID lesen Rebalance/Compound einen leeren Slot.

5.1 Entry ausführen

1. Automations-Liste → Wick&Wait – Entry → Execute → Tx bestätigen.

2. **Erwartung:** eine neue LP-Position; im Portfolio sichtbar. `LP_POSITION_SLOT` enthält jetzt die Token-ID (im Deploy-/Context-Bereich bzw. on-chain prüfbar).

5.2 Auto-Compound prüfen

1. Sicherstellen: Status *Active*, Gas-Deposit gefüllt.
2. Fork-Zeit über das Interval vorrücken und den Keeper laufen lassen:

```
cd packages/contracts && npm run execute:fork  
# oder: npx hardhat run scripts/execute-automations.ts --network localhost
```

Der Keeper führt jede *aktive, public, getriggerte* Automation aus.

3. **Erwartung:** Collect + Fee Deposit + Increase laufen; die Gas-Reserve ist wieder auf `minFeeDeposit` aufgefüllt, die Liquidität der Position steigt; Eintrag in der *Execution History*.

5.3 Rebalance – „persistent“ (soll feuern)

1. Preis dauerhaft aus der Range schieben (Wegwerf-Treiber auf dem Fork):

```
cd packages/contracts  
POSITION_ID=<tokenId> MODE=persistent SIDE=above WINDOW=1800 \  
  BUFFER_TICKS=80 HOLD_STEPS=6 MAX_SWAP=10000 \  
  npx hardhat run scripts/wick-wait-price-driver.ts --network localhost
```

(`WINDOW` = dein TWAP-Fenster in Sekunden, hier 30 min = 1800. Das Skript wärmt auch das Oracle auf.)

2. In der Liste zeigt die Trigger-Spalte der Rebalance-Automation *met* (grün), sobald der TWAP draußen ist.
3. Keeper laufen lassen (`npm run execute:fork`).
4. **Erwartung:** Position wird geschlossen, neu gesized und um den neuen Preis geöffnet; `LP_POSITION_SLOT` trägt eine neue ID, `LAST_REBALANCE_SLOT` den aktuellen Zeitstempel.

Cooldown beachten: direkt nach einem Rebalance feuert der Trigger `Cooldown` lang nicht erneut – auch wenn der Preis draußen ist. Zum schnellen Testen kleinen Cooldown wählen.

5.4 Rebalance – „wick“ (soll NICHT feuern)

1. Preis kurz raus und innerhalb von `W` zurückschnappen:

```
POSITION_ID=<tokenId> MODE=wick SIDE=above WINDOW=1800 \  
  BUFFER_TICKS=80 HOLD_STEPS=6 MAX_SWAP=100000000 \  
  npx hardhat run scripts/wick-wait-price-driver.ts --network localhost
```

2. **Erwartung:** Der TWAP bleibt (bzw. konvergiert zurück) *IN range*; Trigger-Spalte bleibt *nicht erfüllt*; der Keeper macht nichts. Genau das ist die Wick-Robustheit.

Erfolgskriterien gesamt: Entry mintet & füllt den Slot · Auto-Compound erhöht die Liquidität · persistenter Ausbruch löst genau einen Rebalance aus (dann Cooldown) · kurzer Wick löst nichts aus.

11 · Troubleshooting

Symptom	Ursache / Lösung
Step „Wick & Wait“ / „Swap to Range Ratio“ fehlt im <code>+ Add Step</code>	Contracts nicht deployed/geseedet → <code>deploy-defi-actions.ts</code> + <code>pnpm db:seed</code> .
Rebalance-Trigger feuert nie, obwohl Preis draußen	Pool-Oracle hat zu wenig History für <code>W</code> (Condition revertet) → Cardinality erhöhen / Oracle aufwärmen. Oder Cooldown noch aktiv.
<code>mapGraphToRaw is not a function</code> beim Deploy	Veralteter Vite-Cache → Frontend-Dev-Server neu starten (Cache leeren).
Rebalance/Compound werden nie ausgeführt	Automation nicht <i>Active</i> , kein Gas-Deposit, oder kein Keeper läuft (<code>pnpm execute:fork</code>).
Position landet schief / einseitig	<i>Range Width</i> in Swap-to-Range-Ratio ≠ Mint. Beide auf identisches <code>±%</code> setzen.
<code>Delete</code> ausgegraut	Public-Automation erst <i>Deactivate</i> , dann löschen.

12 · Anhang – Feld-Referenz pro Step

Die Feld-Titel entsprechen 1:1 dem Editor (schema-getrieben). Slot-Felder zeigen einen Picker für deine Context-Variablen; Betrags-Felder haben einen „Volles Guthaben“-Toggle.

Step	Felder (Editor-Titel)
Swap to Range Ratio	Token A · Token B · Fee Tier · Range Width (±%)
LP Mint	Token A · Token B · Fee Tier · Price Range (Preset/Explizit) · Amount A · Amount B · Position Token-ID to Context Slot
Wick & Wait Rebalance	Position Token-Id Slot · TWAP Window · Rebalance Cooldown · Last-Rebalance Slot
Decrease Liquidity	Position Token-ID from Context Slot · Percentage to Remove
Collect	Position Token-ID from Context Slot
Increase Liquidity	Token A · Token B · Position Token-ID from Context Slot · Amount A · Amount B
Fee Deposit	Fee Registry · Token · Top-up Amount (Toggle „Bis Zielreserve auffüllen“)
Interval Condition	Interval (Wert + Einheit) · Start Time

Erzeugt für den manuellen Test der Wick-&-Wait-Strategie · Werte sind Beispiele (BSC-Fork, BTCB/USDT, fee 500).